

Universidad de La Habana
Facultad de Matemática y Computación



Sistema web para el autorreporte de Eventos Supuestamente Atribuibles a la Vacunación o Inmunización (ESAVI) en el Instituto Finlay de Vacunas (IFV)

Autor:

Jabel Resendiz Aguirre

Tutores:

MsC. Joanna Campbell Amos

MsC. Aracelys García Armenteros

Trabajo de Diploma
presentado en opción al título de
Licenciado en Ciencia de la Computación

27 de mayo de 2026

github.com/JabelResendiz/thesis

Capítulo 1

Detalles de Implementación

1.1. Introducción

Este capítulo describe los principales aspectos relacionados con la implementación del sistema web desarrollado para el autorreporte y la gestión operativa de ESAVI. El contenido aborda las tecnologías y herramientas incorporadas durante el desarrollo, la organización estructural de la solución y los mecanismos implementados tanto en el backend como en el frontend.

La implementación materializa las decisiones arquitectónicas definidas en el capítulo anterior mediante una arquitectura modular basada en los principios de Clean Architecture y Domain-Driven Design. La solución integra mecanismos de autenticación, comunicación asíncrona, persistencia de datos y validación de información, con el propósito de garantizar escalabilidad, mantenibilidad y seguridad.

Además, el capítulo expone los principales componentes funcionales de la plataforma, incluyendo el registro de reportes de ESAVI, la gestión de revisiones médicas, la asignación de responsables y el sistema de notificaciones. Finalmente, el contenido aborda elementos asociados al despliegue y la organización técnica de la aplicación.

1.2. Tecnologías y herramientas utilizadas

1.2.1. Herramientas de infraestructura y despliegue

Docker. La contenedorización de la aplicación empleó **Docker** para garantizar la consistencia entre los entornos de desarrollo, pruebas y producción. La documentación oficial de Docker establece que los contenedores empaquetan el código junto con todas sus dependencias, lo que asegura que el software funcione de manera uniforme en cualquier infraestructura [1]. Esta característica elimina el clásico problema de “en mi máquina funciona” y facilita la distribución del proyecto.

Railway. El despliegue de la API, la base de datos MySQL y los servicios externos como RabbitMQ recayó en **Railway**, una plataforma como servicio que permite aprovisionar infraestructura en la nube y conectar el despliegue directamente con un repositorio de GitHub [2]. Railway gestiona de forma automática las variables de entorno, los volúmenes de datos y la exposición de puertos, y activa una nueva construcción y publicación con cada cambio en la rama principal, lo que reduce significativamente el esfuerzo operativo durante el desarrollo [3].

Vercel. Para el *frontend*, el despliegue recayó en **Vercel**, una plataforma optimizada para aplicaciones web modernas que ofrece integración continua con repositorios de GitHub y distribución global mediante una red de entrega de contenido [4].

Es importante señalar que tanto Railway como Vercel constituyen soluciones de desarrollo y demostración, adoptadas exclusivamente para facilitar que el equipo de investigadores del Instituto Finlay de Vacunas (IFV) y los tutores del proyecto pudieran observar los avances de forma remota y en tiempo real. La arquitectura del sistema está diseñada para migrar y contenerizar en servidores físicos dedicados del IFV cuando el proyecto transite a la etapa de producción, garantizando así el control institucional sobre los datos y la infraestructura.

1.2.2. Herramientas de apoyo al desarrollo

Visual Studio Code. El desarrollo del proyecto transcurrió en **Visual Studio Code**, con las extensiones oficiales para C# y Docker. Microsoft recomienda este entorno para el desarrollo en .NET, ya que ofrece una integración completa con el SDK y la CLI de .NET [5]. Dicha integración permite crear, compilar y ejecutar aplicaciones directamente desde el editor, lo que agiliza el flujo de trabajo [6].

Git y GitHub. El control de versiones recayó con **Git** y el alojamiento del código en **GitHub**. GitHub proporciona un historial completo de cambios donde es posible consultar qué modificaciones ocurrieron, quién las hizo y por qué [7]. Al tratarse de un sistema distribuido, cada copia local del repositorio contiene el historial completo del proyecto, lo que permite recuperar cualquier estado anterior del código sin depender del servidor central [8].

Swagger (Swashbuckle). Para documentar y probar la API, el proyecto incorporó **Swagger** a través de la librería Swashbuckle, que genera automáticamente una interfaz web con todos los endpoints, sus parámetros y permite ejecutar peticiones de prueba directamente desde el navegador [9].

1.3. Estructura general de la solución

1.3.1. Organización de proyectos

1.3.2. Separación por capas y responsabilidades

1.3.3. Estructura modular del sistema

1.4. Implementación del backend

1.4.1. Capa de Dominio

La capa de dominio constituye el núcleo del sistema y no depende de ningún otro proyecto de la solución ni de paquetes externos. Contiene las entidades de negocio, los objetos de valor, los enumerados, los eventos de dominio y las reglas intrínsecas del dominio de farmacovigilancia. Su independencia garantiza que los cambios en la infraestructura o en la presentación no afecten la lógica central del negocio.

Entidades y agregados

El diseño de las entidades parte del modelo de base de datos definido en el Capítulo 3. Cada tabla se modela como una clase de C# que hereda de una jerarquía de clases base diseñada para unificar propiedades comunes y permitir la creación de interfaces genéricas en la capa de aplicación.

La jerarquía de herencia parte de `BasicEntity`, una clase abstracta que define las marcas de tiempo comunes a todas las entidades del sistema:

```
1 public abstract class BasicEntity
2 {
3     public DateTime CreatedAt { get; set; }
4     public DateTime UpdatedAt { get; set; }
5 }
```

Ejemplo de código 1.1: Clase base `BasicEntity` con marcas de tiempo

Para contemplar entidades con distintos tipos de identificador, el dominio define la interfaz genérica `IEntity<TId>`:

```
1 public interface IEntity<TId>
2 {
3     TId Id { get; init; }
4 }
```

Ejemplo de código 1.2: Interfaz genérica para identificadores de entidad

A partir de esta interfaz, el dominio proporciona dos clases abstractas especializadas. La clase `GuidEntity` constituye la norma para la mayoría de las entidades del sistema y utiliza un `Guid` como identificador:

```
1 public abstract class GuidEntity : BasicEntity, IEntity<Guid>
2 {
3     public Guid Id { get; init; } = new Guid();
4 }
```

Ejemplo de código 1.3: Clase `GuidEntity` para entidades con identificador `Guid`

Para entidades de catálogo que requieren un identificador entero —como las listas de provincias y municipios, cuyos registros son pocos, estables y no requieren las propiedades de seguridad de un `Guid`—, existe la clase `CatalogEntity`.

```
1 public abstract class CatalogEntity : BasicEntity, IEntity<int>
2 {
3     public int Id { get; init; }
4 }
```

Ejemplo de código 1.4: Clase `CatalogEntity` para entidades con identificador entero

La elección de `Guid` como tipo de identificador para las entidades principales responde a criterios de seguridad y escalabilidad. La documentación oficial de Microsoft establece que un `Guid` (Globally Unique Identifier) es un número pseudoaleatorio de 128 bits con una probabilidad de colisión tan baja que resulta estadísticamente insignificante [10]. A diferencia de los identificadores enteros autoincrementales, un `Guid` no revela la cantidad de registros en una tabla y no es predecible, lo que refuerza la seguridad de la aplicación. Esta propiedad permite, por ejemplo, exponer el identificador en una URL sin riesgo de que un usuario malintencionado itere sobre valores consecutivos.

Gracias a que `BasicEntity` unifica las propiedades comunes `CreatedAt` y `UpdatedAt`, la capa de aplicación puede definir interfaces genéricas que operan sobre cualquier entidad del sistema sin necesidad de implementaciones específicas por cada tipo. El siguiente fragmento, extraído de la capa de aplicación, ilustra esta ventaja:

```
1 public interface IGenericRepository<T> where T : BasicEntity
2 {
3     Task<T> GetByIdAsync<TId>(TId elementId);
4     Task DeleteByIdAsync<TId>(TId elementId);
5 }
```

Ejemplo de código 1.5: Interfaz genérica que opera sobre cualquier entidad que herede de `BasicEntity`

La restricción `where T : BasicEntity` garantiza en tiempo de compilación que la interfaz solo reciba entidades que posean las marcas de tiempo `CreatedAt` y `UpdatedAt`, independientemente del tipo de su identificador. Así, el sistema puede

incorporar nuevas entidades —ya sea con `Guid` o con `int` como identificador— sin modificar las interfaces genéricas existentes, en cumplimiento del Principio de Abierto/Cerrado de SOLID.

Entidades `User` y `Role` Para la gestión de usuarios y roles, el proyecto emplea las clases `IdentityUser` y `IdentityRole` del paquete `Microsoft.AspNetCore.Identity`. Esta decisión evita implementar desde cero funcionalidades críticas como el almacenamiento seguro de contraseñas, la generación de tokens de verificación y el bloqueo de cuentas tras intentos fallidos.

La entidad `User` hereda de `IdentityUser<Guid>` y extiende el modelo con los campos requeridos por el dominio:

```
1 public class User : IdentityUser<Guid>
2 {
3     public Guid RoleId { get; set; }
4     public DateTime CreatedAt { get; set; }
5     public DateTime UpdatedAt { get; set; }
6 }
```

Ejemplo de código 1.6: Entidad `User` que hereda de `IdentityUser<Guid>`

Aunque la clase derivada solo declara tres propiedades adicionales, la herencia de `IdentityUser<Guid>` aporta un conjunto completo de campos que cubren todas las necesidades de gestión de identidad [11]. `UserName` almacena el nombre de usuario único para el inicio de sesión. `Email` contiene el correo electrónico empleado para notificaciones y verificación de cuenta. `PasswordHash` guarda el resultado del algoritmo de hashing y nunca contiene la contraseña en texto plano. `PhoneNumber` registra el teléfono para autenticación de dos factores o recuperación de cuenta. `EmailConfirmed` indica si el usuario verificó su correo electrónico. `LockoutEnd` establece la fecha y hora hasta la cual la cuenta permanece bloqueada tras superar el máximo de intentos fallidos. `AccessFailedCount` lleva el contador de intentos fallidos consecutivos. `SecurityStamp` almacena un valor aleatorio que cambia al modificar credenciales, lo que invalida automáticamente sesiones y tokens anteriores.

La entidad `Role` hereda de `IdentityRole<Guid>` y mantiene la misma coherencia en el tipo de identificador:

```
1 public class Role : IdentityRole<Guid>{ }
```

Ejemplo de código 1.7: Entidad `Role` que hereda de `IdentityRole<Guid>`

La herencia de `IdentityRole<Guid>` proporciona las propiedades `Name` (nombre único del rol) y `NormalizedName` (versión normalizada para búsquedas). La relación entre usuarios y roles sigue una restricción de uno a muchos: cada usuario posee un único rol mediante la clave foránea `RoleId` en la entidad `User`, mientras que un mismo rol puede agrupar a múltiples usuarios. Esta decisión simplifica la administración de

permisos y refleja los perfiles definidos en el diseño del sistema (administrador, médico evaluador y jefe de vacunación municipal).

El uso de `Guid` como tipo de identificador en `IdentityUser` y `IdentityRole` mantiene la coherencia con la clase `GuidEntity` definida para el resto de las entidades del sistema, lo que permite que las tablas de identidad compartan el mismo tipo de clave primaria que las tablas de negocio.

Mecanismo de hashing de contraseñas La documentación oficial de Microsoft describe que el `PasswordHasher` de Identity implementa el algoritmo PBKDF2 (Password-Based Key Derivation Function 2), estándar del RFC 2898 que deriva una clave segura a partir de una contraseña [12]. El algoritmo toma la contraseña del usuario, la combina con un *salt* criptográficamente aleatorio de 128 bits y aplica repetidamente la función HMAC-SHA256¹ durante 100,000 iteraciones como valor predeterminado [14]. Cada iteración adicional duplica el costo computacional de un ataque por fuerza bruta sin afectar perceptiblemente el tiempo de verificación para un usuario legítimo. El formato del hash generado produce una cadena con la siguiente estructura [15]:

`Base64(salt).Base64(subkey)`

Donde el *salt* es un valor aleatorio único por cada contraseña y el *subkey* es el resultado de aplicar PBKDF2 durante el número configurado de iteraciones.

Este mecanismo garantiza tres propiedades de seguridad esenciales:

- **Unicidad del hash:** dos usuarios con la misma contraseña producen hashes completamente diferentes porque el *salt* generado para cada uno es distinto. Esto impide que un atacante compare hashes para identificar contraseñas idénticas dentro de una base de datos comprometida [16].
- **Irreversibilidad computacional:** PBKDF2 es una función unidireccional. El elevado número de iteraciones convierte el cálculo del hash en un proceso deliberadamente costoso en tiempo de CPU, lo que ralentiza significativamente cualquier intento de ataque por fuerza bruta incluso si el atacante posee hardware especializado [14].

¹HMAC-SHA256: código de autenticación de mensajes basado en hash [13]. Funciona aplicando de forma anidada el algoritmo SHA-256 a la combinación de una clave secreta (o *salt*) con el mensaje. Internamente, la clave se procesa mediante una operación XOR con una secuencia de relleno interna (*ipad*) antes de aplicar el primer hash al mensaje; posteriormente, el resultado se combina mediante XOR con una secuencia externa (*opad*) y se somete a un segundo hash SHA-256, garantizando simultáneamente la integridad y autenticidad del resultado.

- **Invalidación de sesiones previas:** la propiedad `SecurityStamp`, generada aleatoriamente y modificada cada vez que el usuario cambia su contraseña, se incluye en la generación de tokens de autenticación. Si un atacante obtiene credenciales antiguas, el cambio de `SecurityStamp` invalida inmediatamente cualquier sesión o token previo [17].

La verificación de la contraseña durante el inicio de sesión se realiza internamente mediante el método `VerifyHashedPassword`, que recalcula el hash con el *salt* almacenado y compara los resultados en tiempo constante, sin que la aplicación necesite conocer el texto plano de la contraseña en ningún momento [14].

Generación de tokens para verificación de identidad El paquete Identity incluye proveedores de tokens integrados que generan cadenas criptográficamente seguras para distintos propósitos. El sistema emplea estos tokens durante el proceso de registro para verificar que un usuario controla la dirección de correo electrónico declarada y para permitir el establecimiento seguro de su contraseña inicial. La documentación oficial explica que el proveedor predeterminado `DataProtectorTokenProvider` utiliza internamente la API de Protección de Datos de ASP.NET Core, la cual aplica cifrado autenticado (AES-256-CBC² con HMAC-SHA256) sobre los datos del token [19]. Esto garantiza tres propiedades de seguridad esenciales: el token no revela información del usuario si es interceptado, cualquier modificación invalida la firma y provoca el rechazo del servidor, y un atacante no puede generar un token válido sin acceder a las claves de protección del servidor [20].

El token posee una vigencia predeterminada de 24 horas, controlada por la propiedad `TokenLifespan` de `DataProtectionTokenProviderOptions`, transcurrida la cual expira automáticamente [21]. La validación ocurre en el servidor sin necesidad de consultar una tabla de tokens pendientes, lo que simplifica la arquitectura y elimina la necesidad de limpiar tokens expirados. El caso de uso concreto que orquesta la generación y validación de estos tokens durante el registro de médicos evaluadores se detalla en la sección dedicada a la capa de aplicación.

Configuración de seguridad adicional Además de los valores predeterminados, el proyecto configura el número de iteraciones del algoritmo PBKDF2 a 600,000 mediante la clase `PasswordHasherOptions`:

```
1 builder.Services.Configure<PasswordHasherOptions>(options =>
```

²AES-256-CBC (Advanced Encryption Standard con modo Cipher Block Chaining y clave de 256 bits): algoritmo de cifrado simétrico estandarizado por el Instituto Nacional de Estándares y Tecnología (NIST, por sus siglas en inglés), agencia del gobierno de Estados Unidos. Transforma datos legibles en texto cifrado usando una clave secreta, y el modo CBC garantiza que mensajes idénticos produzcan resultados cifrados diferentes [18].


```
2 {  
3     options.IterationCount = 600_000;  
4 };
```

Ejemplo de código 1.8: Configuración de iteraciones de hashing en Program.cs

Este ajuste sigue las recomendaciones actuales de OWASP para el algoritmo PBKDF2-SHA256, que establecen un mínimo de 600,000 iteraciones para aplicaciones desplegadas en entornos de producción [16]. El incremento respecto al valor predefinido de 100,000 iteraciones aumenta el costo computacional de cada intento de verificación de contraseña en un factor de seis, lo que dificulta aún más los ataques de diccionario sin afectar perceptiblemente el tiempo de inicio de sesión de un usuario legítimo.

Objetos de valor

El dominio incluye el objeto de valor **IdentityNumber**, que encapsula la validación del formato del carné de identidad cubano, la extracción de la fecha de nacimiento y el cálculo de la edad. Esta decisión responde a que varias entidades del sistema —**VaccinatedSubject**, **Reporter** y **MedicalReviewer**— contienen un carné de identidad y requieren las mismas operaciones sobre él. Al extraer esta lógica en un objeto de valor inmutable, se evita la duplicación de código y se garantiza que cualquier validación o cálculo sobre el carné se aplique de manera uniforme en todo el sistema.

Enumerados

Los enumerados representan clasificaciones propias del negocio cuyos valores no cambian durante la ejecución de la aplicación. El dominio los define mediante el tipo **enum** de C#, lo que garantiza que toda la aplicación comparta las mismas categorías sin riesgo de discrepancias entre capas. Estos tipos no constituyen entidades ni tablas en la base de datos porque sus valores son fijos y no requieren persistencia independiente.

En contraste, conceptos como **Province** y **Municipality** sí se modelan como entidades y tablas. Existe una relación entre ambos —cada municipio pertenece a una provincia— que resulta más clara de expresar mediante llaves foráneas que mediante un enumerado extenso. Además, el sistema realiza operaciones frecuentes de búsqueda y filtrado sobre estos datos, para las cuales una tabla indexada ofrece mejor rendimiento. Por último, la lista de provincias y municipios puede actualizarse ocasionalmente por cambios administrativos, y una tabla permite modificarla sin recompilar la aplicación.

Eventos de dominio

Los eventos de dominio representan sucesos relevantes del negocio que otras partes del sistema deben conocer. El proyecto los define como clases simples con propiedades que identifican los datos del suceso y los utiliza para desencadenar el envío de notificaciones por correo electrónico sin acoplar la lógica de envío a las entidades. Esta arquitectura mantiene el dominio aislado de los detalles de infraestructura y permite agregar nuevos manejadores sin modificar las entidades ni los casos de uso.

Durante la implementación surgieron además dos entidades de infraestructura no previstas en el diseño inicial: **RefreshToken**, que permite la renovación de sesiones sin reautenticación, y **AuditLog**, que registra las operaciones realizadas sobre el sistema con fines de trazabilidad. Ambas heredan de **GuidEntity** y su definición detallada se presenta en el Anexo E.

1.4.2. Capa de Aplicación

Implementación del patrón CQRS

Casos de uso y servicios de aplicación

Validaciones y manejo de solicitudes

Mapeo y transformación de datos

1.4.3. Capa de Infraestructura

Persistencia de datos

Configuración de Entity Framework Core

Migraciones y acceso a datos

Integración con servicios externos

Comunicación asíncrona mediante mensajería

1.4.4. Capa de Presentación

Diseño de la API REST

Controladores y endpoints

Documentación de la API

Manejo de excepciones y respuestas

1.4.5. Seguridad de la aplicación

Autenticación basada en JWT y Refresh Token

El sistema emplea JSON Web Tokens (JWT) como mecanismo de autenticación sin estado, combinado con tokens de actualización (refresh tokens) almacenados en cookies HttpOnly. Esta combinación responde a dos objetivos contrapuestos: el token de acceso debe tener una vida útil corta para limitar el daño en caso de robo, pero el usuario no debe verse obligado a iniciar sesión repetidamente.

JSON Web Token (JWT) Un JWT es un estándar abierto definido en el RFC 7519 que permite transmitir información verificable entre dos partes mediante un token firmado digitalmente [22]. El token consta de tres secciones codificadas en Base64 y separadas por puntos: encabezado, carga útil y firma. La carga útil contiene los *claims* —afirmaciones sobre el usuario, como su identificador, nombre y rol—, mientras que la firma garantiza que el token no fue alterado desde su emisión.

El sistema genera el JWT mediante la clase `JwtTokenGenerator`, que utiliza HMAC-SHA256 como algoritmo de firma con una clave secreta simétrica almacenada en variables de entorno. Esta clave nunca aparece en el código fuente ni se expone al cliente.

```
1 public async Task<string> GenerateToken(User user)
2 {
3     var key = new SigningCredentials(
4         new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_jwtSettings
5             .Secret)),
6         SecurityAlgorithms.HmacSha256);
7
8     var role = await _userManager
9         .GetRolesAsync(user)
10        .FirstOrDefault();
11
12    var claims = new List<Claim>
13    {
14        new Claim(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
15        new Claim(JwtRegisteredClaimNames.GivenName, user.UserName!),
16
17        new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().
18            ToString()),
19        new Claim(ClaimTypes.Role, role!)
20    };
21
22    var securityToken = new JwtSecurityToken(
23        signingCredentials: key,
24        issuer: _jwtSettings.Issuer,
25        audience: _jwtSettings.Audience,
26        expires: DateTime.UtcNow.AddMinutes(_jwtSettings.
27            ExpiryMinutes),
28        claims: claims);
29
30    return new JwtSecurityTokenHandler().WriteToken(securityToken);
31 }
```

Ejemplo de código 1.9: Generación del JWT en `JwtTokenGenerator`

El método define cuatro claims en el token: identificador del usuario (`Sub`), nombre de usuario (`GivenName`), identificador único del token (`Jti`) y rol del usuario (`Role`). La expiración se configura en 15 minutos mediante `JwtSettings.ExpiryMinutes`.

El middleware de ASP.NET Core valida cada solicitud entrante verificando cuatro propiedades del token: que el emisor y la audiencia coincidan con los valores configurados, que la firma HMAC-SHA256 sea correcta y que el token no haya expirado. Si un atacante intercepta el JWT, puede usarlo durante un máximo de 15 minutos, transcurridos los cuales el token expira y el middleware lo rechaza automáticamente.

Refresh Token en cookie HttpOnly Para compensar la corta vida del JWT sin sacrificar seguridad, el sistema implementa refresh tokens. Un refresh token es un valor aleatorio generado criptográficamente que permite obtener nuevos tokens de acceso sin que el usuario vuelva a ingresar sus credenciales. El método `GenerateRefreshToken` utiliza la clase `RandomNumberGenerator` de .NET, una alternativa criptográficamente segura frente a generadores predecibles como `System.Random` [23].

```
1 private string GenerateRefreshToken()
2 {
3     using var rng = RandomNumberGenerator.Create();
4     var randomBytes = new byte[64];
5     rng.GetBytes(randomBytes);
6     return Convert.ToBase64String(randomBytes);
7 }
```

Ejemplo de código 1.10: Generación criptográfica del refresh token

El método produce 64 bytes de entropía criptográfica y los codifica en Base64. Este valor se almacena en la base de datos junto con su fecha de vencimiento y el identificador del usuario.

La seguridad de este mecanismo descansa en tres decisiones reflejadas en la configuración de la cookie que transporta el refresh token:

```
1 Response.Cookies.Append("refreshToken", authResult.RefreshToken,
2     new CookieOptions
3     {
4         HttpOnly = true,
5         Secure = true,
6         SameSite = SameSiteMode.None,
7         Expires = DateTime.UtcNow.AddHours(_jwtSettings.RefreshTokenExpiryHours)
8     });
```

Ejemplo de código 1.11: Configuración de la cookie del refresh token

- `HttpOnly = true`: impide que JavaScript acceda al valor de la cookie, lo que neutraliza los ataques de cross-site scripting (XSS), ataque que inyecta código JavaScript malicioso en una página web legítima para robar datos del usuario [24].
- `Secure = true`: restringe la transmisión de la cookie a conexiones HTTPS, evitando su exposición en redes no cifradas.
- `SameSite = None`: permite el envío de la cookie desde dominios distintos al del backend —necesario cuando frontend y backend residen en servidores separados—, pero el estándar obliga a combinarlo con `Secure = true` [25]. Esta

configuración se complementa con una política CORS (Cross-Origin Resource Sharing) que restringe los dominios autorizados a realizar solicitudes a la API.

- **Expires:** establece la caducidad de la cookie en 8 horas mediante el valor definido en las variables de entorno.

En el entorno de producción previsto para el Instituto Finlay, donde ambos componentes compartirán el mismo dominio, la política CORS dejará de ser necesaria y `SameSite` podrá restringirse a `Lax`, lo que reforzará la seguridad al impedir el envío de la cookie en solicitudes cross-site.

Además, cada vez que el cliente utiliza un refresh token para obtener un nuevo JWT, el sistema aplica rotación: revoca el token anterior y emite uno nuevo. Si un atacante logra robar un refresh token, la rotación limita su utilidad a una única renovación, tras la cual el token legítimo queda invalidado y el usuario deberá iniciar sesión nuevamente.

Envío del token en cada solicitud El cliente recibe el JWT en el cuerpo de la respuesta de inicio de sesión y lo almacena en memoria. En cada solicitud subsiguiente, lo incluye en el encabezado `Authorization: Bearer <token>`, conforme al estándar definido en el RFC 6750 [26]. El refresh token, por su parte, viaja automáticamente en la cookie sin intervención del código frontend, lo que reduce la superficie de ataque al no exponerlo a lógica JavaScript.

Autorización basada en roles

Una vez que el middleware de autenticación valida el JWT y establece la identidad del usuario, el sistema delega el control de acceso en dos mecanismos complementarios: el atributo `[Authorize]` para restringir endpoints por rol, y un servicio de contexto de usuario para recuperar la identidad del usuario autenticado dentro de la capa de aplicación.

El atributo `[Authorize(Roles = "SectionResponsible")]` indica al middleware de autorización que solo los usuarios cuyo claim `Role` coincida con ese valor pueden ejecutar el método. Si el token no contiene el rol requerido, el middleware retorna automáticamente un código 403 (Forbidden) con un mensaje JSON, según la configuración establecida en el evento `OnForbidden` del JWT Bearer.

Para que la capa de aplicación pueda acceder a la identidad del usuario autenticado sin que el frontend la envíe manualmente en cada solicitud, el sistema implementa `UserContextService`. Este servicio extrae el claim `NameIdentifier` del token JWT validado —disponible en el `HttpContext` de la solicitud— y lo expone mediante el método `GetUserId()`:

```
1 public class UserContextService : IUserContextService
2 {
3     private readonly IHttpContextAccessor _httpContextAccessor;
4
5     public UserContextService(IHttpContextAccessor
6     httpContextAccessor)
7     {
8         _httpContextAccessor = httpContextAccessor;
9     }
10
11     public Guid GetUserId()
12     {
13         var user = _httpContextAccessor.HttpContext?.User
14         ?? throw new UnauthorizedAccessException("No
15         authenticated user.");
16
17         var userId = user.FindFirst(ClaimTypes.NameIdentifier)?.
18         Value
19         ?? throw new UnauthorizedAccessException("User ID not
20         found in token.");
21
22         return Guid.Parse(userId);
23     }
24 }
```

Ejemplo de código 1.12: Servicio de contexto de usuario autenticado

El método `GetUserId()` obtiene el `HttpContext` de la solicitud actual mediante `IHttpContextAccessor` y recupera el claim `ClaimTypes.NameIdentifier` —cuyo valor se estableció durante la generación del JWT con `JwtRegisteredClaimNames.Sub`—. Este enfoque elimina la necesidad de que cada endpoint reciba explícitamente el identificador del usuario como parámetro, lo que reduce el riesgo de suplantación: el identificador no puede ser manipulado por el cliente porque proviene del token validado por el servidor.

La capa de aplicación consume este servicio para personalizar las consultas según el usuario autenticado. El siguiente fragmento muestra cómo un revisor médico obtiene únicamente los reportes que le fueron asignados, sin recibir como parámetro su propio identificador:

```
1 public async Task<PagedResultDto<ReportMedicalReviewerDto>>
2     GetReportAssignment(
3         PagedRequestDto paged,
4         ReportMedicalReviewerFilter filter)
5 {
6     var userId = _userContextService.GetUserId();
7
8     var medicalReviewer = await _unitOfWork
9     .GetRepository<MedicalReviewer>()
```

```
9         .FirstOrDefaultAsync(sr => sr.UserId == userId)
10         ?? throw new UnauthorizedAccessException("User is not a
    medical reviewer");
11
12     var reportIds = _unitOfWork
13         .GetRepository<MedicalReviewAssignment>()
14         .GetAllByItems(mra =>
15             mra.MedicalReviewerId == medicalReviewer.Id &&
16             mra.Status == ReviewAssignmentStatus.Pending)
17         .Select(a => a.AefiReportId)
18         .Distinct();
19
20     var query = _unitOfWork
21         .GetRepository<AefiReport>()
22         .GetAllByItems(r => reportIds.Contains(r.Id));
23
24     // Filtrado, paginación y proyección a DTO...
25 }
```

Ejemplo de código 1.13: Uso de `UserService` en la capa de aplicación

El método obtiene el identificador del usuario desde el token, lo utiliza para localizar al revisor médico asociado y filtra los reportes cuyas asignaciones coincidan con ese revisor. En ningún momento el frontend envía el identificador del usuario; este se deriva exclusivamente del token validado, lo que garantiza que un usuario autenticado solo pueda acceder a sus propios recursos.

Protección de credenciales y contraseñas

Validación de datos y protección ante ataques comunes

Limitación de velocidad

La limitación de velocidad (en inglés, *rate limiting*) restringe el número de solicitudes que un cliente puede realizar a la API en un período de tiempo determinado. Su propósito es proteger el sistema contra ataques de fuerza bruta, enumeración de recursos y denegación de servicio [27]. Además, evita que picos de tráfico legítimo o bucles accidentales en el frontend consuman todos los recursos del servidor y degraden la experiencia del resto de usuarios.

Los endpoints públicos del sistema ya cuentan con CAPTCHA como primera barrera, según se describió anteriormente. El rate limiting actúa como segunda barrera: si un atacante resuelve el CAPTCHA para cada intento, el límite de solicitudes por minuto hace inviable cualquier ataque masivo. Para los endpoints protegidos con JWT, el rate limiting constituye la defensa principal contra el abuso por parte de usuarios autenticados o tokens comprometidos.

Limitación por IP y por sesión El sistema aplica un limitador global que contabiliza todas las solicitudes de un mismo cliente, independientemente del endpoint al que se dirijan. Para clientes anónimos —como un reportante que envía un formulario público—, el límite se aplica por dirección IP: 100 solicitudes por minuto. Para usuarios autenticados, el límite se aplica por identificador de sesión extraído del JWT: 200 solicitudes por minuto. Esta diferenciación evita que usuarios legítimos que comparten red —como el personal de un mismo centro de salud— resulten bloqueados por el comportamiento de un tercero, y otorga mayor margen a quienes ya superaron la barrera de autenticación. Una solicitud solo se procesa si el limitador global lo permite.

Políticas complementarias por tipo de operación Sobre esta base, el sistema añade políticas específicas que restringen aún más ciertos endpoints según la criticidad de la operación. Cada política define un límite de solicitudes, una ventana de tiempo y un comportamiento de encolamiento. La ventana fija restablece el contador al cumplirse el intervalo; la ventana deslizante distribuye el límite en segmentos para suavizar los picos de tráfico legítimo. El encolamiento permite retener las solicitudes excedentes durante un breve período antes de rechazarlas, lo que evita que un pico puntual de actividad legítima genere errores inmediatos al cliente [27].

- **Auth:** 3 solicitudes por minuto para endpoints de autenticación, sin encolamiento. Un usuario legítimo inicia sesión pocas veces al día; tres intentos por minuto bastan para la operación normal y hacen inviable un ataque de fuerza bruta incluso si el atacante resuelve el CAPTCHA en cada intento.
- **Command:** 5 solicitudes por minuto para operaciones de escritura estándar, como la creación de reportes ESAVI, con cola de hasta 5 solicitudes. Equivale a un reporte cada 12 segundos, ritmo suficiente para un centro de salud con múltiples reportantes simultáneos.
- **ReportDaily:** 30 solicitudes por día para la creación de reportes desde una misma IP, mediante ventana deslizante de 24 horas dividida en 24 segmentos. Cubre la actividad diaria de un centro de salud activo y acota el daño máximo de un ataque automatizado a 30 reportes falsos, los cuales pueden identificarse y eliminarse manualmente.
- **CommandDaily:** 20 solicitudes por día para operaciones administrativas como la asignación de reportes o la finalización de revisiones médicas. Refleja el ritmo real de trabajo de un jefe de vacunación municipal o médico evaluador.
- **Query:** 60 solicitudes por minuto para operaciones de lectura, con ventana deslizante dividida en 3 segmentos y cola de hasta 10 solicitudes. Las consultas no

modifican el estado del sistema; una consulta por segundo permite una navegación fluida mientras dificulta la extracción masiva de datos.

El siguiente fragmento muestra la configuración de dos de estas políticas dentro de la clase `RateLimitingMiddleware`:

```

1 // Autenticación: muy restrictivo
2 options.AddFixedWindowLimiter("Auth", config =>
3 {
4     config.PermitLimit = 3;
5     config.Window = TimeSpan.FromMinutes(1);
6     config.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
7     config.QueueLimit = 0;
8 });
9
10 // Comandos estándar: restrictivo
11 options.AddFixedWindowLimiter("Command", config =>
12 {
13     config.PermitLimit = 5;
14     config.Window = TimeSpan.FromMinutes(1);
15     config.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
16     config.QueueLimit = 5;
17 });

```

Ejemplo de código 1.14: Configuración de políticas de rate limiting

Aplicación a endpoints El atributo `[EnableRateLimiting]` aplica las políticas sobre controladores completos o endpoints específicos. Un mismo endpoint puede combinar múltiples políticas cuando requiere protección en distintas ventanas de tiempo. Por ejemplo, el controlador de autenticación declara únicamente `Auth` a nivel de clase:

```

1 [ApiController]
2 [Route("api/[controller]")]
3 [EnableRateLimiting("Auth")]
4 public class AuthenticationController : ControllerBase { }

```

Ejemplo de código 1.15: Aplicación del rate limiting a controladores y endpoints

El endpoint de creación de reportes, en cambio, combina dos políticas: `Command` limita la tasa por minuto y `ReportDaily` impone un límite diario por IP. Ambas se evalúan en conjunto con el limitador global: la solicitud solo se procesa si las tres capas lo permiten.

```

1 [HttpPost]
2 [EnableRateLimiting("Command")]
3 [EnableRateLimiting("ReportDaily")]
4 public async Task<IActionResult> Create(CreateReportDto dto) { }

```

Ejemplo de código 1.16: Combinación de políticas en un mismo endpoint

Bloqueo manual por administrador del servidor Aun con las barreras descritas, persiste un escenario que las defensas automáticas no pueden resolver por sí solas: un atacante que distribuya sus solicitudes entre múltiples direcciones IP, que opere desde el extranjero sin relación legítima con el sistema cubano de farmacovigilancia, o que mantenga su actividad por debajo de los umbrales configurados para no activar ninguna alarma. La detección de estas amenazas requiere criterio humano y herramientas a nivel de infraestructura.

Esta responsabilidad recae en el administrador del servidor del Instituto Finlay, no en los usuarios de la aplicación. Mediante el análisis de logs, reglas de firewall y filtrado por geolocalización, el equipo de TI puede identificar patrones sospechosos y denegar el acceso desde regiones no autorizadas o direcciones IP específicas. Esta separación de responsabilidades mantiene la aplicación enfocada en la lógica de farmacovigilancia, mientras que la seguridad perimetral queda en manos de quienes administran el servidor físico donde el sistema se despliega en producción.

1.5. Implementación del frontend

1.5.1. Arquitectura del frontend

1.5.2. Gestión del estado y comunicación con la API

1.5.3. Construcción de interfaces de usuario

1.5.4. Implementación de formularios

1.5.5. Validaciones y experiencia de usuario

1.5.6. Manejo de autenticación en el cliente

1.5.7. Diseño responsivo

1.6. Implementación de funcionalidades principales

1.6.1. Registro de reportes de ESAVI

1.6.2. Gestión de revisiones médicas

1.6.3. Asignación de responsables

1.6.4. Gestión de causalidad

1.6.5. Sistema de notificaciones

1.6.6. Visualización y consulta de reportes

1.7. Despliegue de la solución

1.7.1. Contenerización de servicios

1.7.2. Configuración del entorno de despliegue

1.7.3. Publicación de la aplicación

1.8. Conclusiones del capítulo

En este capítulo se presentaron los principales detalles relacionados con la implementación de la plataforma desarrollada, describiendo las tecnologías empleadas, la

organización arquitectónica de la solución y los mecanismos utilizados en el backend y el frontend. Además, se abordaron aspectos asociados a la seguridad, la integración entre componentes y el despliegue del sistema, evidenciando la materialización práctica de la propuesta presentada en el capítulo anterior.

Capítulo 2

resultados

Apéndice A

Resultados de farmacovigilancia pasiva en Cuba y las Américas

A.1. Reporte de RAM notificadas por farmacovigilancia pasiva en Cuba

En el Anexo A.1 se presenta una figura con la evolución de las notificaciones de reacciones adversas a medicamentos (RAM) en Cuba entre 2014 y 2020. Estos datos corresponden al sistema de **farmacovigilancia pasiva**, basado en la notificación espontánea de sospechas de RAM por parte del personal sanitario.

Año	Número de reportes RAM	Tasa de reporte por millón de habitantes
2014	20 380	1812
2015	20 611	1833
2016	18 977	1688
2017	17 070	1519
2018	14 736	1312
2019	15 982	1425
2020	13 025	1163

Fuente: Base de datos nacional de farmacovigilancia

Figura A.1: Notificaciones de reacción adversa a medicamentos (RAM) declaradas mediante farmacovigilancia pasiva en Cuba (2014-2020). Fuente: CECMED, Informe Anual 2020 [28].

A.2. Reporte según grupo farmacológico

En el Anexo A.2 se incluye una figura que muestra los reportes de reacción adversa a medicamentos (RAM) según el grupo farmacológico, a partir de la farmacovigilancia pasiva del año 2020.

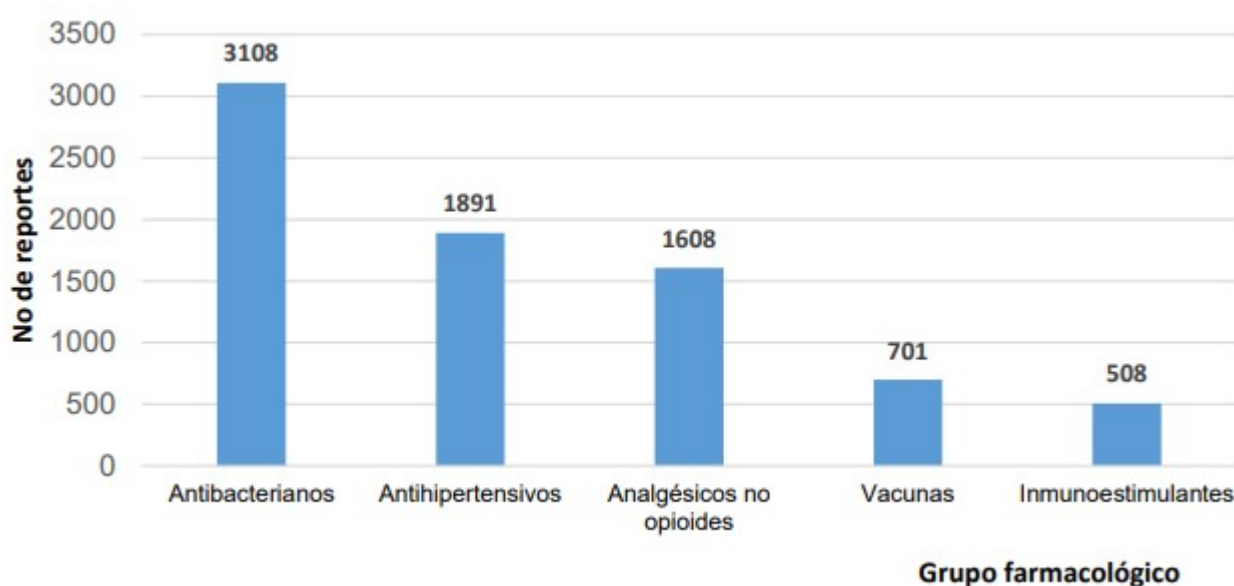


Figura A.2: Reporte de reacción adversa a medicamentos (RAM) según grupo farmacológico. Farmacovigilancia pasiva, 2020. Adaptado de [28]

A.3. Notificaciones de ESAVI por países de la región de las Américas

En el Anexo A.3 se incluye una figura que presenta el número de ESAVI notificados por los países de la región de las Américas.



Figura A.3: Número de ESAMI notificados por los países de la región de las Américas. Adaptado de [29]

A.4. Estructura del Sistema de Vigilancia de Medicamentos en Cuba

En el Anexo A.4 se incluye una figura que presenta la organización del sistema cubano de vigilancia de medicamentos.

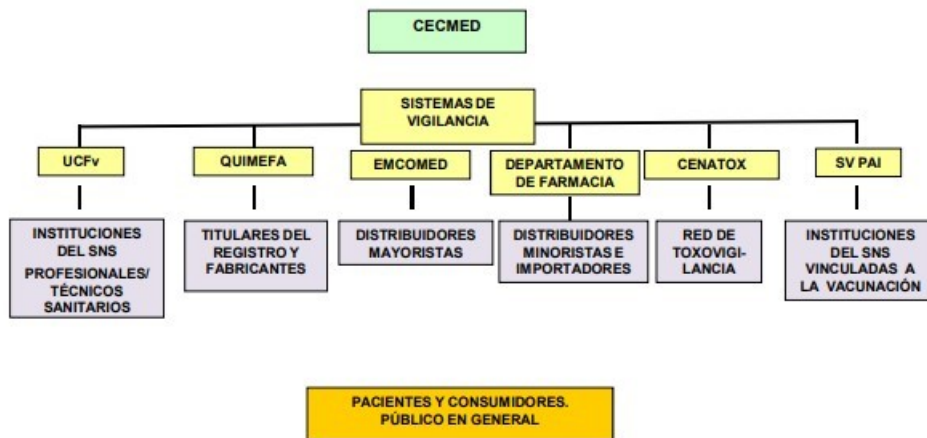


Figura A.4: Estructura del Sistema de Vigilancia de Medicamentos en Cuba. Adaptado de: [30]

El sistema de vigilancia de medicamentos en Cuba se estructura bajo la rectoría del CECMED, autoridad reguladora nacional encargada de la evaluación, control y toma de decisiones sobre la seguridad de los medicamentos a partir de la información generada en el sistema.

Para el cumplimiento de estas funciones, se articula una red de vigilancia integrada por diversos componentes que interactúan de manera coordinada (ver Figura A.4) [30]:

- **UCNFV (MINSAP):** Cohesiona la Red de Farmacoepidemiología en los 169 municipios del país y actúa como el canal central para la notificación espontánea de profesionales y pacientes, siendo las vacunas la excepción gestionada de forma especializada por el PAI [30, 31].
- **Fabricantes y Titulares de Registro (QUIMEFA):** Involucra a las 24 empresas farmacéuticas nacionales, donde cada una cuenta con personal designado para la vigilancia y reporta directamente sobre la seguridad y calidad de sus productos [30].
- **Distribución Mayorista (EMCOMED):** Coordina la vigilancia en las 24 droguerías del país, detectando problemas de calidad durante el almacenamiento y transportación [30].
- **Distribución Minorista (Departamento de Farmacia):** Supervisa la vigilancia en las más de 2,000 farmacias comunitarias y hospitalarias del Sistema Nacional de Salud [30].
- **CENATOX:** Lidera la red de toxicovigilancia, supervisando las intoxicaciones y reacciones adversas de naturaleza tóxica [30].
- **SV PAI:** Realiza la vigilancia especializada de vacunas mediante el monitoreo de los ESAVI, bajo la supervisión del Viceministerio de Higiene y Epidemiología [30].

Toda la información captada por estos efectores fluye hacia el Observatorio de Vigilancia Postcomercialización del CECMED, donde se evalúa la relación beneficio-riesgo para emitir alertas farmacológicas, retirar lotes o modificar registros sanitarios en tiempo real [30].

Apéndice B

Modelos oficiales de notificación en Cuba

B.1. Modelo 33-36-02

En el Anexo B.1 se incluye el **Modelo 33-36-02**, el formulario oficial utilizado por los profesionales de la salud en Cuba para la notificación de sospechas de reacciones adversas a medicamentos dentro del sistema nacional de farmacovigilancia.

Notificación de sospechas de Reacciones Adversas a Medicamentos

Informe Confidencial

1. Por favor notifique todas las reacciones a los fármacos recientemente introducidos en el mercado y todas las reacciones moderadas, severas, raras y mortales a todos los medicamentos, productos estomatológicos, quirúrgicos, DIU, suturas y lentes de contacto.
2. Indique el/los medicamentos a que se atribuyen los efectos nocivos causados al paciente (en caso de existir más de uno marque con un asterisco el nombre del más sospechoso). Indique además todos los medicamentos administrados los tres meses anteriores.
3. No deje de notificar reacciones adversas raras por considerar que no estén relacionadas con el medicamento, estas pueden ser las más importantes.
4. En caso de anomalías congénitas, aunque estas fueran menores, indique todos los medicamentos tomados por la madre durante la gestación. En el acápite patologías que presenta el paciente poner las patologías maternas crónicas y las padecidas durante la gestación, sin olvidar las etiologías infecciosas.
5. Debe poner en observaciones especiales cualquier dato de interés que crea importante incluir, incluyendo procederes que hubo de indicársele al paciente para el tratamiento de la reacción, hábitos tóxicos, resultados de exámenes complementarios, etc.
6. Trate siempre de llenar la planilla completa, sin omitir nada, pero NO DEJE DE NOTIFICAR PORQUE LE FALTE ALGUN DATO.

ESTA PLANILLA DEBE ENVIARSE A LA FARMACIA PRINCIPAL MUNICIPAL QUE LA HARA LLEGAR A LA UNIDAD PROVINCIAL DE FARMACOVIGILANCIA UBICADA EN LA VICEDIRECCION DE MEDICAMENTOS DE LA DIRECCION PROVINCIAL DE SALUD.

Cualquier consulta o ampliación de la información contactar con:

Departamento de Farmacoepidemiología

Unidad Coordinadora Nacional de Farmacovigilancia

MINSAP 23 y N 7mo piso Vedado

Teléfono: 839-63-60

Email: giset@msp.sld.cu, ismary@msp.sld.cu

Figura B.2: Instructivo en el dorso del modelo 33-36-02: Notificación de sospechas de Reacciones Adversas a Medicamentos por el Profesional Sanitario, Adaptado de [31]

B.2. Modelo 33-36-03

En el Anexo B.2 se incluye el **Modelo 33-36-03**, el formulario oficial utilizado por los pacientes en Cuba para la notificación de sospechas de reacciones adversas a medicamentos dentro del sistema nacional de farmacovigilancia.

Unidad que reporta:				Provincia				Municipio																				
PACIENTE:																												
Nombre y Apellidos _____																												
Edad: _____				Sexo: F ☐ M ☐		Color de la Piel: P ☐ M ☐ N ☐ O ☐																						
Nombre del que reporta: _____				Médico ☐ Lic. Farmacia ☐ Téc. Farmacia ☐ Lic. Enfermería ☐ Enfermera ☐ Estomatólogo ☐ Otro ☐																								
Medicamentos tomados hasta 3 meses antes de la RAM. Marcar con una cruz los sospechosos		Lote	Vía de Administración	Dosis diaria	TRATAMIENTO						Motivo de Prescripción																	
					Inicio			Fin																				
					Día	Mes	Año	Día	Mes	Año																		
Fabricante: _____				Patologías que presenta el paciente _____																								
REACCIONES. Enumérense por separado				Fecha de Inicio			Fecha de Término			Requirió ingreso o prolongó su estadía hospitalaria		Atención de Urgencia	Puso en peligro su vida															
				Día	Mes	Año	Día	Mes	Año	SI	NO	SI	NO															
				<table border="1" style="width: 100%; border-collapse: collapse;"> <tr> <td></td> <td style="text-align: center;">SI</td> <td style="text-align: center;">NO</td> </tr> <tr> <td>¿Se ha suspendido la medicación?</td> <td></td> <td></td> </tr> <tr> <td>¿Ha mejorado al suspenderla?</td> <td></td> <td></td> </tr> <tr> <td>¿Se administró nuevamente este medicamento?</td> <td></td> <td></td> </tr> <tr> <td>¿Si se administró nuevamente, hubo recurrencia de síntomas?</td> <td></td> <td></td> </tr> </table>											SI	NO	¿Se ha suspendido la medicación?			¿Ha mejorado al suspenderla?			¿Se administró nuevamente este medicamento?			¿Si se administró nuevamente, hubo recurrencia de síntomas?		
	SI	NO																										
¿Se ha suspendido la medicación?																												
¿Ha mejorado al suspenderla?																												
¿Se administró nuevamente este medicamento?																												
¿Si se administró nuevamente, hubo recurrencia de síntomas?																												
				<table border="1" style="width: 100%; border-collapse: collapse;"> <tr> <th colspan="2" style="text-align: center;">DESENLACE</th> </tr> <tr> <td>Recuperado</td> <td style="text-align: center;">☐</td> </tr> <tr> <td>No recuperado</td> <td style="text-align: center;">☐</td> </tr> <tr> <td>Recuperado con secuela</td> <td style="text-align: center;">☐</td> </tr> <tr> <td>Mortal</td> <td style="text-align: center;">☐</td> </tr> </table>										DESENLACE		Recuperado	☐	No recuperado	☐	Recuperado con secuela	☐	Mortal	☐					
DESENLACE																												
Recuperado	☐																											
No recuperado	☐																											
Recuperado con secuela	☐																											
Mortal	☐																											
Observaciones adicionales:																												
INSTRUCTIVO AL DORSO												Fecha de Notificación: _____ <table border="1" style="width: 100%; border-collapse: collapse;"> <tr> <td></td> <td></td> <td></td> </tr> <tr> <td>Día</td> <td>Mes</td> <td>Año</td> </tr> </table>					Día	Mes	Año									
Día	Mes	Año																										

Figura B.3: Modelo 33-36-03: Notificación de sospechas de Reacciones Adversas a Medicamentos por el Paciente. Adaptado de [31]

Notificación de sospechas de Reacciones Adversas a Medicamentos
Informe Confidencial

1. Un efecto no deseado producido por medicamentos es un efecto que produce una incomodidad o daño al paciente, que ocurre después de consumir uno o más medicamentos siendo utilizados éstos a la dosis correcta para prevenir o tratar una enfermedad.
2. Por favor notifique todos los efectos no deseados a cualquier medicamento, productos estomatológicos, quirúrgicos, dispositivos intrauterinos, suturas y lentes de contacto.
3. Indique el/los medicamentos a que se atribuyen los efectos no deseados causados al paciente (en caso de existir más de uno marque con una cruz el nombre del más sospechoso). Indique además todos los medicamentos administrados los tres meses anteriores poniendo uno en cada línea.
4. En la columna para qué se lo indicó el médico ponga el motivo en la línea correspondiente a cada medicamento.
5. Cuando el que notifica es la persona que sufrió el efecto no deseado marque con una cruz la casilla Paciente, si se refiere a alguna otra persona, pues marca Persona que reporta.
6. La casilla Cantidad al día se refiere a la cantidad de medicamento (tableta, cucharada, inyección etc.) que ingirió en un día.
7. Los efectos no deseados deben ser escritos por separado en cada línea, poniendo en primer lugar si es posible, el que más molestias le pasó.
8. La casilla tiempo pasado entre el consumo del medicamento e inicio del efecto no deseado se refiere al tiempo transcurrido desde que comenzó a tomar el medicamento y comienza el efecto adverso.
9. No deje de notificar efectos no deseados por considerar que no estén relacionadas con el medicamento, estos pueden ser los más importantes.
10. Añada cualquier dato de interés que crea importante incluir en la casilla que pregunta si quiere decir algo más. Si desea poner el lote del medicamento o el fabricante puede ponerlo en esta casilla.
11. Trate siempre de llenar la planilla completa, sin omitir nada, pero **NO DEJE DE NOTIFICAR PORQUE LE FALTE ALGUN DATO.**

Figura B.4: Instructivo en el dorso del modelo 33-36-03: Notificación de sospechas de Reacciones Adversas a Medicamentos por el Paciente. Adaptado de [31]

B.3. Modelo 84-30-2

En el Anexo B.3 se incluye el **Modelo 84-30-2**, el formulario oficial utilizado por los profesionales de la salud en Cuba para la notificación de sospechas de eventos supuestamente atribuibles a la vacunación o inmunización (ESAVI) dentro del sistema nacional de farmacovigilancia.

Apéndice C

Base de Datos Central de Farmacovigilancia (FarmaVigiC)

C.1. Campos de la base de datos central de farmacovigilancia (FarmaVigiC)

En el Anexo C.1 se describen los campos de la base de datos central de farmacovigilancia (FarmaVigiC) utilizada en Cuba.

Tabla C.1: Campos de la base de datos central de farmacovigilancia (FarmaVigiC).
Elaboración propia a partir de [31] (p. 42).

Grupos campos	de	Variables o campos
Paciente		Nombre y apellidos Edad Sexo Raza Antecedentes patológicos personales Requirió ingreso Requirió atención de urgencia Requirió reposo Fue baja laboral o escolar Peligró su vida

Grupos de campos	Variables o campos
Notificador	Unidad que reporta Provincia Municipio Nombre y apellidos Especialidad
Medicamentos	Medicamento sospechoso Presentación cantidad Presentación unidad Forma farmacéutica Fabricante Diluyente Lote Vía de administración Dosis cantidad Dosis unidad Dosis frecuencia Fecha de inicio del tratamiento Fecha fin del tratamiento Continuación Motivo de prescripción Otro fármaco 1 Otro fármaco 2 Otro fármaco 3 Otro fármaco 4 Otro fármaco 5 Otro fármaco 6 Vía, dosis, lote de otros fármacos Fecha de administración de otros fármacos

Grupos de campos	Variables o campos
Reacciones adversas	Reacción principal Otra RAM 1 Otra RAM 2 Otra RAM 3 Otra RAM 4 Otra RAM 5 Otra RAM 6 Fecha de inicio RAM Fecha fin de RAM Continuación
Clasificación	Grupo de edades Sexo Nivel de atención Especialidad Grupo farmacológico Sistema de órgano afectado Severidad Causalidad Importancia Frecuencia
Desenlace	Suspendió la medicación Supresión positiva Re exposición Recurrencia de síntomas Desenlace Recuperado No recuperado Recuperado con secuela
Organización e información adicional	Mes de entrada a la base Secuencia temporal Observaciones Comentarios Resultados de necropsia Fecha de notificación
Evaluación de la calidad	E1 (evalúa al notificador) E2 (evalúa al revisor) E3 (evalúa calidad del llenado de la base)

- A nivel municipal se trabajan 71 campos
- A nivel provincial se trabajan 72 campos
- A nivel nacional se trabajan 75 campos

Apéndice D

Formularios de referencia

D.1. Formulario VAERS - Octubre 2025

En el Anexo D.1 se incluye el formulario oficial del Vaccine Adverse Event Reporting System (VAERS) correspondiente a octubre de 2025. Este formulario constituye el instrumento estándar utilizado para la notificación de eventos adversos posteriores a la vacunación en los Estados Unidos.

VAERS Vaccine Adverse Event Reporting System www.vaers.hhs.gov		Adverse events are possible reactions or problems that occur during or after vaccination. Items 2, 3, 4, 5, 6, 17, 18 and 21 are ESSENTIAL and should be completed. Patient identity is kept confidential. <u>Instructions</u> are provided on the last two pages.	
INFORMATION ABOUT THE PATIENT WHO RECEIVED THE VACCINE (Use Continuation Page if needed)			
1. Patient name: (first) _____ (last) _____ Street address: _____ City: _____ State: _____ County: _____ ZIP code: _____ Phone: () _____ Email: _____		9. Prescriptions, over-the-counter medications, dietary supplements, or herbal remedies being taken at the time of vaccination: _____ 10. Allergies to medications, food, or other products: _____	
2. Date of birth: (mm/dd/yyyy) _____ 3. Sex: ☐ Male ☐ Female		11. Other illnesses at the time of vaccination and up to one month prior: _____	
4. Date and time of vaccination: (mm/dd/yyyy) _____ Time: h:mm _____ AM ☐ PM ☐		12. Chronic or long-standing health conditions: _____	
5. Date and time adverse event started: (mm/dd/yyyy) _____ Time: h:mm _____ AM ☐ PM ☐			
6. Age at vaccination: Years _____ Months _____ 7. Today's date: (mm/dd/yyyy) _____			
8. Pregnant at time of vaccination?: ☐ Yes ☐ No ☐ Unknown (If yes, describe the event, any pregnancy complications, and estimated due date if known in item 18)			
INFORMATION ABOUT THE PERSON COMPLETING THIS FORM		INFORMATION ABOUT THE FACILITY WHERE VACCINE WAS GIVEN	
13. Form completed by: (name) _____ Relation to patient: ☐ Healthcare professional/staff ☐ Patient (yourself) ☐ Parent/guardian/caregiver ☐ Other: _____ Street address: _____ ☐ Check if same as item 1 City: _____ State: _____ ZIP code: _____ Phone: () _____ Email: _____		15. Facility/clinic name: _____ Fax: () _____ Street address: _____ ☐ Check if same as item 13 City: _____ State: _____ ZIP code: _____ Phone: () _____	
14. Best doctor/healthcare professional to contact about the adverse event: Name: _____ Phone: () _____ Ext: _____		16. Type of facility: (Check one) ☐ Doctor's office, urgent care, or hospital ☐ Pharmacy or store ☐ Workplace clinic ☐ Public health clinic ☐ Nursing home or senior living facility ☐ School or student health clinic ☐ Home ☐ Other: _____ ☐ Unknown	
WHICH VACCINES WERE GIVEN? WHAT HAPPENED TO THE PATIENT?			
17. Enter all vaccines given on the date listed in item 4: (Route is HOW vaccine was given, Body site is WHERE vaccine was given)			
Vaccine (type and brand name)	Manufacturer	Lot number	Route
select	select	select	select
select	select	select	select
select	select	select	select
select	select	select	select
18. Describe the adverse event(s), treatment, and outcome(s), if any: (symptoms, signs, time course, etc.) Use Continuation Page if needed		21. Result or outcome of adverse event(s): (Check all that apply) ☐ Doctor or other healthcare professional office/clinic visit ☐ Emergency room/department or urgent care ☐ Hospitalization: Number of days (if known) _____ Hospital name: _____ City: _____ State: _____ ☐ Prolongation of existing hospitalization (vaccine received during existing hospitalization) ☐ Life threatening illness (immediate risk of death from the event) ☐ Disability or permanent damage ☐ Patient died - Date of death: (mm/dd/yyyy) _____ ☐ Congenital anomaly or birth defect ☐ None of the above	
19. Medical tests and laboratory results related to the adverse event(s): (include dates) Use Continuation Page if needed			
20. Has the patient recovered from the adverse event(s)?: ☐ Yes ☐ No ☐ Unknown			
ADDITIONAL INFORMATION			
22. Any other vaccines received within one month prior to the date listed in item 4:			
Vaccine (type and brand name)	Manufacturer	Lot number	Route
select	select	select	select
select	select	select	select
23. Has the patient ever had an adverse event following any previous vaccine?: (If yes, describe adverse event, patient age at vaccination, vaccination dates, vaccine type, and brand name) ☐ Yes ☐ No ☐ Unknown		24. Patient's race: ☐ American Indian or Alaska Native ☐ Asian ☐ Black or African American ☐ Native Hawaiian or Other Pacific Islander ☐ White ☐ Unknown ☐ Other: _____	
25. Patient's ethnicity: ☐ Hispanic or Latino ☐ Not Hispanic or Latino ☐ Unknown		26. Immuniz. proj. report number: (Health Dept use only) _____	
COMPLETE ONLY FOR U.S. MILITARY/DEPARTMENT OF DEFENSE (DoD) RELATED REPORTS			
27. Status at vaccination: ☐ Active duty ☐ Reserve ☐ National Guard ☐ Beneficiary ☐ Other: _____		28. Vaccinated at Military/DoD site: ☐ Yes ☐ No	

FORM FDA VAERS 2.0 (10/25)

SAVE

Figura D.1: Formulario VAERS - Octubre 2025

D.2. Formulario v-safe - Enero 2021

En el Anexo D.2 se incluyen capturas del sistema v-safe del CDC correspondientes a enero de 2021. Este sistema constituye una herramienta de seguimiento activo utilizada para monitorizar síntomas y eventos adversos posteriores a la vacunación.

Let's start today's health check-in.

How are you feeling today? *



Good



Fair



Poor

Next >

Your information in v-safe is protected by administrative, technical, and physical measures that safeguard the confidentiality, integrity, and privacy of personal information. To the extent v-safe uses existing information systems managed by CDC, FDA, and other federal agencies, the systems employ strict security measures appropriate for the level of sensitivity of the data. [Learn more about v-safe.](#)

Figura D.2: Estado general

Fever Check

Have you had a fever or felt feverish today? *

☐ Yes
☐ No

< Previous

Next >

Your information in v-safe is protected by administrative, technical, and physical measures that safeguard the confidentiality, integrity, and privacy of personal information. To the extent v-safe uses existing information systems managed by CDC, FDA, and other federal agencies, the systems employ strict security measures appropriate for the level of sensitivity of the data. [Learn more about v-safe.](#)




Figura D.3: Verificación de fiebre

Symptom Check

Have you had any of these symptoms where you got the shot (injection site)? *

Select all that apply.

☐ Pain
☐ Redness
☐ Swelling
☐ Itching
☐ None

Have you experienced any of these symptoms today? *

Select all that apply.

☐ Chills
☐ Headache
☐ Joint pains
☐ Muscle or body aches
☐ Fatigue or tiredness
☐ Nausea
☐ Vomiting
☐ Diarrhea
☐ Abdominal pain
☐ Rash, not including the immediate area around the injection site
☐ None

Any other symptoms or health conditions you want to report

< Previous

Next >

Your information in v-safe is protected by administrative, technical, and physical measures that safeguard the confidentiality, integrity, and privacy of personal information. To the extent v-safe uses existing information systems managed by CDC, FDA, and other federal agencies, the systems employ strict security measures appropriate for the level of sensitivity of the data. [Learn more about v-safe.](#)

Figura D.4: Verificación de síntomas

Health Impact

Did any of the symptoms or health conditions you reported today cause you to: *

Select all that apply.

☐ Be unable to work
☐ Be unable to do your normal daily activities
☐ Get care from a doctor or other healthcare professional
☐ None of the above

< Previous

Submit

Your information in v-safe is protected by administrative, technical, and physical measures that safeguard the confidentiality, integrity, and privacy of personal information. To the extent v-safe uses existing information systems managed by CDC, FDA, and other federal agencies, the systems employ strict security measures appropriate for the level of sensitivity of the data. [Learn more about v-safe.](#)




Figura D.5: Impacto en la salud

Figura D.6: Capturas del sistema v-safe para el seguimiento de síntomas posteriores a la vacunación

Apéndice E

Entidades de infraestructura

E.1. RefreshToken

La entidad `RefreshToken` permite la renovación de sesiones sin que el usuario deba iniciar sesión nuevamente. Cuando un usuario se autentica, el sistema genera un token de acceso JWT de corta duración y un token de actualización que persiste en esta tabla. Al expirar el token de acceso, el cliente presenta el token de actualización y obtiene uno nuevo.

```
1 public class RefreshToken : GuidEntity
2 {
3     public string Token { get; set; } = null!;
4     public DateTime ExpiresAt { get; set; }
5     public bool Revoked { get; set; }
6     public Guid UserId { get; set; }
7     public User User { get; set; } = null!;
8 }
```

Ejemplo de código E.1: Entidad `RefreshToken`

`Token` almacena el valor del token de actualización. `ExpiresAt` define su vencimiento. `Revoked` permite invalidarlo antes de esa fecha. `UserId` y `User` asocian cada token con su usuario.

E.2. AuditLog

La entidad `AuditLog` registra las operaciones realizadas sobre el sistema y constituye la base para las funcionalidades de auditoría y trazabilidad descritas en la sección 1.4 del Capítulo 4. Contempla tanto usuarios autenticados como reporteros anónimos.

Tabla E.1: Campos de la entidad AuditLog

Campo	Tipo	Descripción
UserId	Guid?	Usuario autenticado (nulo si es anónimo)
UserEmail	string?	Correo del usuario autenticado
ReporterName	string?	Nombre del reportero anónimo
ReporterEmail	string?	Correo del reportero anónimo
Action	string	Operación: Create, Update, Delete, View
EntityName	string	Entidad afectada (ej. AefiReport)
EntityId	Guid?	Identificador del registro afectado
Details	string?	Descripción legible de la acción
OldValues	string?	Datos antes del cambio (JSON)
NewValues	string?	Datos después del cambio (JSON)
IpAddress	string?	Dirección IP de origen
CreatedAt	DateTime	Fecha y hora de la operación

Glosario

ESAVI Evento Supuestamente Atribuible a la Vacunación o Inmunización. Cualquier evento adverso que se sospecha relacionado con la administración de una vacuna. 1, 24–26

notificación espontánea Sistema de reporte voluntario en el que profesionales de la salud, pacientes o cualquier persona pueden informar sobre sospechas de reacciones adversas a medicamentos sin necesidad de una solicitud formal. 26

reacción adversa a medicamentos (RAM) Según la OMS, reacción nociva y no deseada que se presenta tras la administración de un medicamento, a dosis utilizadas habitualmente en la especie humana, para prevenir, diagnosticar o tratar una enfermedad, o para modificar cualquier función biológica". Nótese que esta definición implica una relación de causalidad entre la administración del medicamento y la aparición de la reacción . 23, 24

tóxica Relacionada con sustancias dañinas, venenosas o nocivas para la salud de los organismos vivos. 26

Siglas

CECMED Centro para el Control Estatal de Medicamentos, Equipos y Dispositivos Médicos. 23, 25, 26

CENATOX Centro Nacional de Toxicovigilancia. 26

MINSAP Ministerio de Salud Pública de Cuba. 26

PAI Programa de Atención Integral. 26

UCNFV Unidad Coordinadora Nacional de Farmacovigilancia. 26

Bibliografía

- [1] Docker Inc. *Docker overview*. 2024. URL: <https://docs.docker.com/get-started/overview/> (visitado 24-05-2026) (vid. pág. 1).
- [2] Railway Corp. *Railway Docs*. 2024. URL: <https://docs.railway.com/> (visitado 24-05-2026) (vid. pág. 2).
- [3] Railway Corp. *Deployments*. 2024. URL: <https://docs.railway.com/deployments/> (visitado 24-05-2026) (vid. pág. 2).
- [4] Vercel Inc. *Vercel Documentation*. 2024. URL: <https://vercel.com/docs> (visitado 24-05-2026) (vid. pág. 2).
- [5] Microsoft. *Using .NET in Visual Studio Code*. 2024. URL: <https://code.visualstudio.com/docs/languages/dotnet> (visitado 24-05-2026) (vid. pág. 2).
- [6] Microsoft. *C# for Visual Studio Code*. 2024. URL: <https://marketplace.visualstudio.com/items?itemName=ms-dotnettools.csharp> (visitado 24-05-2026) (vid. pág. 2).
- [7] GitHub, Inc. *About commits*. 2024. URL: <https://docs.github.com/en/pull-requests/committing-changes-to-your-project/creating-and-editing-commits/about-commits> (visitado 24-05-2026) (vid. pág. 2).
- [8] Scott Chacon y Ben Straub. *Pro Git*. Apress, 2014. Cap. Getting Started - About Version Control. URL: <https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control> (visitado 24-05-2026) (vid. pág. 2).
- [9] SmartBear Software. *Swagger UI*. 2024. URL: <https://swagger.io/tools/swagger-ui/> (visitado 24-05-2026) (vid. pág. 2).
- [10] Microsoft. *Guid.NewGuid Method*. Documentación oficial de .NET. Describe la generación de GUIDs Versión 4 con 122 bits de entropía fuerte. 2026. URL: <https://learn.microsoft.com/en-us/dotnet/api/system.guid.newguid> (vid. pág. 4).

- [11] Microsoft. *IdentityUser<TKey>Class*. Documentación oficial de .NET. Representa un usuario en el sistema Identity con propiedades como Email, PasswordHash, SecurityStamp y LockoutEnd. 2024. URL: <https://learn.microsoft.com/en-us/dotnet/api/microsoft.aspnetcore.identity.identityuser-1> (visitado 25-05-2026) (vid. pág. 5).
- [12] B. Kaliski. *PKCS #5: Password-Based Cryptography Specification Version 2.0*. Inf. téc. 2898. Estándar que define PBKDF2. Especifica la derivación de claves a partir de contraseñas mediante la aplicación iterativa de una función pseudoaleatoria (como HMAC-SHA256) combinada con un salt. IETF, 2000. URL: <https://datatracker.ietf.org/doc/html/rfc2898> (visitado 25-05-2026) (vid. pág. 6).
- [13] H. Krawczyk, M. Bellare y R. Canetti. *HMAC: Keyed-Hashing for Message Authentication*. Inf. téc. 2104. Especificación oficial del mecanismo HMAC. Define el algoritmo para autenticación de mensajes mediante funciones hash criptográficas como SHA-256. IETF, 1997. URL: <https://datatracker.ietf.org/doc/html/rfc2104> (visitado 25-05-2026) (vid. pág. 6).
- [14] Microsoft. *PasswordHasher<TUser>Class*. Documentación oficial de ASP.NET Core Identity. Implementa PBKDF2 con HMAC-SHA256, salt de 128 bits y 100,000 iteraciones por defecto para el hashing de contraseñas. 2024. URL: <https://learn.microsoft.com/en-us/dotnet/api/microsoft.aspnetcore.identity.passwordhasher-1> (visitado 25-05-2026) (vid. págs. 6, 7).
- [15] Microsoft. *ASP.NET Core Identity PasswordHasher*. Repositorio oficial en GitHub. Contiene la implementación del formato de hash: Base64(salt).Base64(subkey). 2024. URL: <https://github.com/aspnet/AspNetIdentity/blob/main/src/Microsoft.AspNetCore.Identity.Core/PasswordHasher.cs> (visitado 25-05-2026) (vid. pág. 6).
- [16] OWASP Foundation. *Password Storage Cheat Sheet*. Guía oficial de OWASP para almacenamiento seguro de contraseñas. Recomienda PBKDF2-SHA256 con 600,000 iteraciones para aplicaciones en producción. 2024. URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (visitado 25-05-2026) (vid. págs. 6, 8).
- [17] Microsoft. *SecurityStamp Validation in ASP.NET Core Identity*. Documentación oficial. Explica que el SecurityStamp se regenera al cambiar credenciales e invalida sesiones y tokens anteriores. 2024. URL: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity-configuration> (visitado 25-05-2026) (vid. pág. 7).

- [18] National Institute of Standards and Technology. *Advanced Encryption Standard (AES)*. Inf. téc. FIPS PUB 197. Estándar oficial del NIST que define el algoritmo de cifrado simétrico AES con claves de 128, 192 y 256 bits y el modo de operación CBC. NIST, 2001. URL: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf> (visitado 25-05-2026) (vid. pág. 7).
- [19] Microsoft. *Token providers in ASP.NET Core Identity*. Documentación oficial. Describe los proveedores de tokens para confirmación de correo, restablecimiento de contraseña y autenticación de dos factores. 2024. URL: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity-configuration> (visitado 25-05-2026) (vid. pág. 7).
- [20] Microsoft. *ASP.NET Core Data Protection Overview*. Documentación oficial. Describe la API de protección de datos usada para cifrar y firmar tokens, con gestión automática de claves y rotación. 2024. URL: <https://learn.microsoft.com/en-us/aspnet/core/security/data-protection/introduction> (visitado 25-05-2026) (vid. pág. 7).
- [21] Microsoft. *Configure ASP.NET Core Identity*. Documentación oficial. Cubre la configuración de IdentityOptions: bloqueo, contraseñas, tokens, cookies y validación de SecurityStamp. 2024. URL: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity-configuration> (visitado 25-05-2026) (vid. pág. 7).
- [22] M. Jones, J. Bradley y N. Sakimura. *JSON Web Token (JWT)*. Inf. téc. 7519. Estándar que define la estructura, claims y firma de los JSON Web Tokens. IETF, 2015. URL: <https://datatracker.ietf.org/doc/html/rfc7519> (visitado 25-05-2026) (vid. pág. 10).
- [23] Microsoft. *RandomNumberGenerator Class*. Documentación oficial de .NET sobre la generación de números aleatorios criptográficamente seguros. 2024. URL: <https://learn.microsoft.com/en-us/dotnet/api/system.security.cryptography.randomnumbergenerator> (visitado 25-05-2026) (vid. pág. 12).
- [24] OWASP Foundation. *HttpOnly*. Documentación de OWASP sobre el atributo HttpOnly. Explica cómo impide que JavaScript acceda a las cookies, mitigando ataques XSS. 2024. URL: <https://owasp.org/www-community/HttpOnly> (visitado 26-05-2026) (vid. pág. 12).
- [25] OWASP Foundation. *Secure Cookie Attribute*. Guía de OWASP sobre atributos de seguridad en cookies. Explica HttpOnly como defensa contra XSS, Secure para transmisión exclusiva sobre HTTPS, y SameSite para controlar el envío en solicitudes cross-site. 2024. URL: <https://owasp.org/www-community/controls/SecureCookieAttribute> (visitado 26-05-2026) (vid. pág. 12).

- [26] M. Jones y D. Hardt. *The OAuth 2.0 Authorization Framework: Bearer Token Usage*. Inf. téc. 6750. Estándar que define el uso del encabezado Authorization: Bearer para enviar tokens de acceso. IETF, 2012. URL: <https://datatracker.ietf.org/doc/html/rfc6750> (visitado 25-05-2026) (vid. pág. 13).
- [27] Microsoft. *Rate limiting middleware in ASP.NET Core*. Documentación oficial sobre la configuración del middleware de rate limiting en ASP.NET Core. Describe las políticas de ventana fija, ventana deslizante y limitación por IP. 2024. URL: <https://learn.microsoft.com/en-us/aspnet/core/performance/rate-limit> (visitado 27-05-2026) (vid. págs. 15, 16).
- [28] Centro para el Control Estatal de Medicamentos, Equipos y Dispositivos Médicos. *Informe anual de farmacovigilancia. Cuba, 2020*. Accedido el 3 de mayo de 2026. 2020. URL: https://www.cecmed.cu/sites/default/files/adjuntos/vigilancia/farmacov/FV_para_informe_anual_2020.pdf (vid. págs. 23, 24).
- [29] Organización Panamericana de la Salud. *Manual de vigilancia de eventos supuestamente atribuibles a la vacunación o inmunización en la Región de las Américas*. Versión oficial adaptada en español de la obra original en inglés: Global manual on surveillance of adverse events following immunization (WHO, 2016). Organización Panamericana de la Salud. Washington, D.C., 2021. ISBN: 978-92-75-32386-1. DOI: 10.37774/9789275323861 (vid. pág. 25).
- [30] Celeste Sánchez G. et al. *Vigilancia de medicamentos en Cuba. Desarrollo actual y nuevos retos*. Accedido el 3 de mayo de 2026. 2010. URL: <https://www.paho.org/sites/default/files/CUB-VIGILANCIA-DE-MEDICAMENTOS-EN-CUBA.pdf> (vid. págs. 25, 26).
- [31] Giset Jiménez López e Ismary Alfonso Orta. *Normas y procedimientos de trabajo del sistema cubano de farmacovigilancia*. Accedido el 3 de mayo de 2026. 2012. URL: <https://files.sld.cu/cdfc/files/2012/10/normas-y-procedimientos-2012.pdf> (vid. págs. 26, 28, 29, 31, 32, 35).
- [32] Centro para el Control Estatal de Medicamentos, Equipos y Dispositivos Médicos (CECMED). «Disposición Reguladora No. 00-515». En: *Boletín del CECMED XXVI.00-515* (oct. de 2025). Edición Ordinaria. Consultada la página 22 donde se encuentra el modelo 84-30-2. ISSN: 1684-1832. URL: <https://www.cecmed.cu/sites/default/files/adjuntos/ambitor/AR%20No.%2000-515.pdf> (vid. pág. 34).